

Flatland: Malfunctions research on Train Training

Guillem Gili i Bueno^[0009–0009–7593–2850], Felix Mark Kratzsch

Universität Potsdam, Am Neuen Palais 10, 14469 Potsdam, Germany

Abstract. This report was created for the course Railway Scheduling at Potsdam University. The focus of this report is creating timetables in an unreliable environment with ASP, which is important for many real-world applications. The environment used is Flatland, which implements many aspects of trains as well as malfunctions. To that end, this paper will detail, how malfunctions work in detail and what different types of encoding were implemented, to generate plans for this environment. This will be followed by a short evaluation, which will illustrate how an incremental encoding is well suited to address the challenges of this environment.

Keywords: ASP · Multi-Agent Pathfinding · Resilience planning.

Table of Contents

Flatland: Malfunctions research on Train Training.....	1
<i>Guillem Gili i Bueno, Felix Mark Kratzsch</i>	
1 Introduction.....	2
2 Introduction to Flatland and Malfunction mechanics	3
2.1 Flatland	3
2.2 Particularities of malfunctions	3
2.3 Edge cases.....	4
3 Core malfunctions analysis / particularities.....	4
3.1 Implications on Encodings.....	4
3.2 Our interests	5
4 Solutions	6
4.1 Graph	6
4.2 Naive Graph.....	7
4.3 Incremental.....	7
5 Testing	8
5.1 Environments.....	8
5.2 Benchmarking	9
6 Results	10
7 Conclusions	11
A Appendix	13

1 Introduction

This report is a product of the course Railway Scheduling at Potsdam University. It focuses on finding timetables in a Railway Environment. The goal of the course is stated as ‘pushing the boundary of Flatland’. In particular, the goal of our group this semester was to explore further what concepts were necessary or useful regarding malfunctions within the Flatland Framework.

In this report, a brief introduction to Flatland is made and the particularities surrounding the usage of malfunctions are highlighted. Afterwards we will propose solutions (which are our main contributions to the project), to solve such problems, starting with a baseline and followed by two derivations. One will serve to compare reutilization and recomputation, the other will implement an incremental ASP approach.

As part of the project, we also selected interesting instances, for diverse and comparable scenarios. Our solutions will than be tested against them. It will be followed by a short evaluation, a conclusion and an outlook over the field.

2 Introduction to Flatland and Malfunction mechanics

2.1 Flatland

Flatland[Murphy, 2025a] is the framework this report researches on. To introduce it, let us just quote from the official repository:

Flatland is a railway scheduling challenge hosted by AICrowd that seeks to solve the problem of multi-agent pathfinding for trains in large railway networks. Although approaches across all domains (e.g. reinforcement learning, operations research) are welcome, this repository focuses on integrating ASP-based solutions within the Flatland framework.

The simulation environment allows specific connections between different roads, different train speeds and for unexpected malfunctions. As the title of this report alludes to, we will be focusing on the more malfunction-adjacent implications, and measuring across various aspects that may be related to it.

There are two kinds of horizons in Flatland. Personal horizons, which can be used for optimizing and are train-dependant (e.g. for machine learning approaches), and a global horizon at which the instance will end. We used the horizons as an estimate for our computation windows (see section 3.1), and deactivated the global horizon.

2.2 Particularities of malfunctions

A flatland simulation is run one step at a time, meaning that a train of the max speed 1 (default) will advance to the next cell given a corresponding action. At run-time, a malfunction (or multiple) may happen at any point. Upon a malfunction, affected trains, fail executing their action and have to wait for a set duration. This may invalidate previous solutions; and force trains to arrive later. Any encoding called on a malfunction would have to:

- Figure out the current state, as it only gets, what was forwarded previously
- Rerun the problem solution, inheriting previous choices and adding the malfunctioning implications

Our interface, as is provided by [Murphy, 2025a], allows for up to two encodings to solve malfunctions. A mandatory **primary** encoding, which is run when the simulation starts, to compute the initial plan. It receives the instance information (map information). The **secondary** encoding is optional and, if specified, will run after every malfunction to compute a new plan. It receives only forwarded atoms. Should no secondary encoding be specified, the **primary** is used instead.

For the purpose of forwarding atoms, the **save/1** and **load/1** atoms were added during the semester[Murphy, 2025c]. They are unary predicates and **load** is solely dependent on the previous runs. An alternative would have been a binary time-dependant version, where all previous information is combined with the timesteps when a malfunction happened, always receiving all previously saved information. Nonetheless, the unary predicate was chosen for more flexibility.

2.3 Edge cases

Collision When there is no distance between trains, a malfunctioning train might have another crashing into it (this may happen recursively). All affected trains have their action then fail as well.

Trying to use unmatching actions with prior to a malfunction Even though the simulation may reach the end successfully (with all trains reaching a destination timely) a solving run will be considered invalid if the actions are incompatible with the originally chosen (this means we have to be able to deduce the same previous actions for each train).

Having to delay an end Due to malfunctions, a simulation may not be able to finish, so both the global timestamps and the `end(Agent, Position, Time)` may require a certain amount of delay before being finished.

Auto Spawning Depending on the interface, malfunctions off-map, might lead to them automatically spawning, when the malfunction is done. (For more info see [Murphy, 2025b])

3 Core malfunctions analysis / particularities

3.1 Implications on Encodings

Modularity A common issue when debugging an ASP problem is to figure out which choice rules or deductions may be the biggest timesink in your program. To debug this is usually complicated, and will in all likelihood discourage someone unfamiliar with the particularities of grounding and what is used beyond syntax. To draw a simile, I may not know all the secrets of a C++ compiler; but I can write a mediocre C++ code, with timestamps that allow me to figure what particular part of my program is slow, then iterate and improve. This is the case for most programming languages, but no such straightforward approach exists in *Clingo*.

By being allowed to selectively load a set of atoms (that have been generated dynamically), we can easily identify which atom generation is slowing down the program. Doing this is far from the most common way to debug these issues in your average *Clingo* program, but it does respect the fact that *Clingo* wants the solver to be solution-agnostic.

Window of grounding The mechanics of malfunctions, have made us focus on trying to build an encoding that uses a window, rather than solving a specific instance problem. While the problem-agnostic nature of ASP usually focuses on direct problems that can be grounded and solved; when building an encoding for malfunctions, one realizes that all timestamps prior to the current are not

necessary to reach a successful solution (neither the atoms used to indicate when a train is at a particular position at a particular time). As such one does not need to ground time values outside the window that has been calculated.

This also shows how to work around against the limitations of ASP; in problems where the horizon of events is too large one may just have too much grounding to do for it. Segmenting said problem seems like the obvious workaround around it, and opens up the possibility of unlimitedly running simulations. This definitely jumps outside *Flatland*, but chip microcontrollers often work in an infinite-loop; working in *Flatland* has showed us that windowed ASP could be a good use case.

3.2 Our interests

While a malfunction may not seem too complicated to work on, there is a huge variety of edge cases that can (and will) appear as one tries to write a successful encoding. Not only that, multiple consecutive malfunctions may happen in a simulation, some of them at the same timestamp. This means that an encoding with a ‘leaky grounding’ will not work on more malfunction-prone simulations. As such the first focus was to have a solution that actually worked for all edge cases was an important solution, as well as a solution that used the forementioned window of grounding. This initial solution was named **Graph**¹.

The forwarding of information allows, to pass finished representations instead of recomputing them. For that we implemented a second approach: **Graph_naive**². Opposed to the previous solution, passes as little information as possible and recomputes as much as possible. While this part is usually not the most time-costly part of the NP-natured problems that an ASP encoding solves, it is interesting nonetheless to study whether there are advantages of statically reading rather than calculating. The difference may become relevant when one wants to work on a very big map representation; or when one decides to ground a more complex representation for the sake of simplifying the search space.

Finally, since the simulation itself is completely tied to iteratively going through the time, we found it interesting to study how an incremental solution tied to time would perform. Timesteps are a severe problem for grounding, thus any reduction, would be a good concept. To that end we implement an **Incremental**³ approach, which in theory should perform best, when many malfunctions happen, as it does not require a problematic estimate to expand the time, but can only ground as much as necessary.

We would have been interested in solution-resilience as well, since what a malfunction effectively is does some pruning in the solution space. But seeing as this would be our first usage of an optimize statement on top of the already complex implementation, and that we had opted rather for prolonging the allowed end timestamp in all of our experimentation, we thought it was beyond the scope of our research.

¹ https://github.com/FeignDeath/Train-Training/tree/main/asp/graph_reschedule

² https://github.com/FeignDeath/Train-Training/tree/main/asp/graph_reschedule_naive

³ <https://github.com/FeignDeath/Train-Training/tree/main/asp/incremental>

4 Solutions

Every encoding below uses an estimate of the horizon as window for its computations, which is passed from the flatland environments.

4.1 Graph

Motivation The basic concept here is to convert Flatland instances into an abstract graph representation. The main difficulty lies therein, that in Flatland coordinates are not the only thing constraining movement, but additionally a direction needs to either be tracked as additional information, or be merged into the graph. For this encoding we chose the second approach. To ensure a graph-like structure vertices are not defined by just a cell, but also from where it was reached. This leads to a graph with directed edges and paths modelling the environment. This approach serves as a baseline for comparison, and a foundation to build upon with the other encodings.

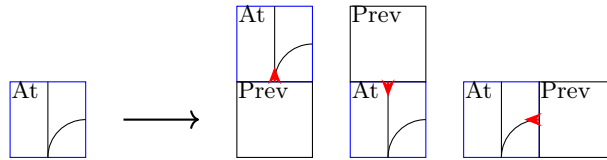


Fig. 1: Cell to Graph

Implementation To that end, the information from flatland is converted similar to Figure 1. The vertex predicates are $\text{vertex}(\text{At}, \text{Prev})$, where **At** is the position of the train, and **Prev** is a possible previous position. As the red arrows show, one could also replace **Prev** with the facing direction. Both contain the same information. With this, one cell is represented by up to 4 vertices. This is handled by introducing a predicate shared resources, to forbid conflicts.

This approach is a rudimentary one, representing agents and their position, for every timestep. As many papers deal with Multi Agent Pathfinding Problems on ASP, I will not go into more detail here, but instead skip to the adaption of malfunctions. (More Info on Solving MAPF with ASP: [Gómez et al., 2020])

The secondary encoding handles malfunctions in three steps.

Reconstruction relies on the previous encoding passing the path and uses it together with the malfunction time, to reconstruct the current state. To that end, the plan is copied for all timesteps up to malfunction time. Except the malfunctioning trains and trains which collide with them. For those the last action fails, therefore one action less is copied.

Persisting computes how long a train malfunctions and derives persistent information which will never be violated even if another malfunction happens in the next timestep. This makes the encoding remember malfunctions and the following information (like train 0 malfunctions there for 5 timesteps), without enforcing them again.

Recompute repeats the pathfinding computation with a shifted window. The window starts at the malfunction timesteps and exceeds the old window by the duration of the worst malfunction at the current timestep. This assumes the worst case, as a malfunction for 5 timesteps, at worst requires 5 additional timesteps to solve the instance.

To save on computation, the secondary encodings does not compute the underlying graph again, but instead gets it passed via save/load, thus receiving the derived atoms as input.

Implications This might lead to an explosion of window size. If for example multiple trains, successively encounter a malfunction of 1 timestep, while not interfering with one another, they would still require one additional timestep to ensure solvability. Instead, this encoding would add as many timesteps as trains, complicating computation.

As this encoding models the paths by timesteps it scales badly with bigger instances, leading to an explosion in grounding.

Lastly, it might not be optimal but given a good guess for the horizon, it can compute a valid path without any optimality quickly.

4.2 Naive Graph

Motivation To ascertain whether, passing the graph-predicates instead of just passing the environment-predicates and recomputing it, improves runtime this encoding was implemented.

Implementation As the secondary encoding receives its sole input from the previous runs it is necessary to load and save the environment predicates on every run. These are then used to compute the graph as well as a solution. Other than that it mirrors the graph approach.

Implications This approach will probably end up being slower than the baseline. As reading the graph from predicates should be slower than computing it. Other than that it should behave quite similarly to the graph approach.

4.3 Incremental

Motivation Incremental encodings as described in [Kaminski et al., 2017] allow for incrementally adding timesteps to the grounding until a problem is solvable. The advantage of that is, if the amount of time required to solve a problem is unknown, this allows to solve it with the minimal amount of timesteps, not

requiring an estimated horizon. The main downside is, that every iteration requires it to prove unsatisfiability before adding the next timestep. Thus, it is mostly applicable, when lacking a good estimate of the horizon.

Nonetheless, this approach should be well suited, for this problem. As instead of being run once, it is run multiple times for every malfunction, where every unnecessary timestep extends the time taken over and over.

Implementation The resulting implementation uses the horizon estimate as the other encodings do, but whenever this leads to unsatisfiability another timestep is added. Thus, still using the given estimate and not having to prove unsatisfiability too often. The incremental implementation is mainly used for malfunction handling. In the case of a malfunction it first tries to solve it, without adding timesteps and only adds them if required.

This required work on the repository side, where *Clingo* is called. Since the repository calls the Python *Clingo* API, it is not possible to call python from *Clingo* again. So Felix needed to modify the given call in `api.py`. The result can be seen in our GitHub Repository and follows closely the implementation in [Kaminski et al., 2017].

Next the encoding was separated into three parts. A **base** program, which would run always. A **step** program, which would only be called if necessary and a **check**, which contains integrity constraints, which need to be checked for the entire program and not just parts of it. The graph generation and pathfinding up to the first horizon was moved to the **base** program. Then the same pathfinding is implemented for the **step** where it only grounds one step at a time. Lastly the goal condition of every agent reaching its goal, was implemented in the **check** part of the program, which is grounded before every solving and removed afterwards.

Representation wise, It is an extension of the graph approach and uses the same building and passing of the graph.

Implications As it solves over and over, this encoding should perform worse in the case of malfunctions, which require the extension of the horizon (additional timesteps).

Other than that it should, outperform the other encodings. This should hold especially for environments dense in malfunctions, where the worst case assumption should add way to many additional timesteps.

5 Testing

5.1 Environments

For testing our encodings we decided on what instances to test on beforehand. We specified three domains with instances with increasing malfunction density (probability of malfunctions)⁴.

⁴ All of our testing instances can be found in https://github.com/FeignDeath/Train-Training/tree/main/envs/testing_instances

The **sparse** domain (Figure 4) consists of 60×60 maps with 6 cities and 6 trains. The original goal, was to do a 100×100 map with 3 trains. But that would simplify the pathfinding and malfunction handling too much, to properly see how the encodings perform. The 6 cities were a necessity as Flatland on a 60×60 map with two cities, often generated maps, which only use a small section of the map, being more akin to a 30×30 map. The interest of this map is mainly to check our graph representation and how it performs with big maps (also requiring many timesteps). These instances should also help magnify the difference between the baseline and the naive encoding, since there are more atoms to be forwarded.

The **dense** domain (Figure 9) consists of 24×24 maps with 3 cities and 20 trains. This is the smallest possible dimension for generation and mainly focuses on malfunction handling. More trains, lead to more malfunctions and require efficient ways of handling them.

The **medium** domain (Figure 5) is a middle ground, of 50×50 , 5 cities and 10 trains.

To ensure comparability, all domains above specify that no more than one rail may connect cities and use malfunctions ranging from 2 to 6 timesteps. This is a compromise of computation. It avoids long malfunctions, which would expand solving time by too much, but still models the real-life unpredictability of them. On the other hand, the one-rail-connection was decided upon in order to enforce more potential conflicts and complicate the instance. Since conflict-solving is the strong side of Answer Set Solving, it makes the instances more challenging, without complicating computation.

5.2 Benchmarking

All Benchmarks were run on cores of an AMD Ryzen 5 7600X (5.45GHz) with 4800Mhz RAM. The instances were run synchronously, with one instance not being allowed to run for more than one hour. The upper bound of memory required per instance was 14 GiB.⁵

⁵ This was achieved by modifying solve.py to return all Clingo stats and create a CSV entry of those and the environment stats after every run.

6 Results

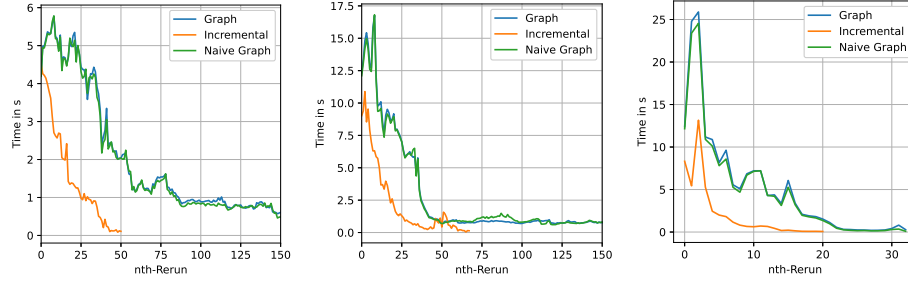


Fig. 2: Time cost over rerun (from left to right sparse, medium, dense)

As visible in Figure 2 our hypothesis, that the graph approach might have problems with an escalating window, did not hold true. The figures show, the time cost for all reruns, averaged over all sparse, medium and dense instances.

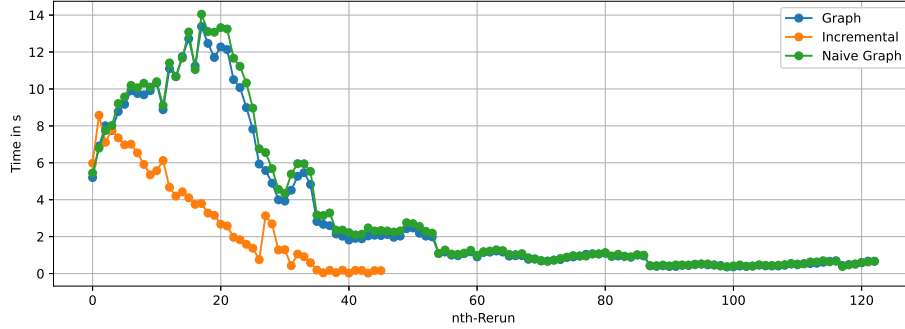


Fig. 3: A singular medium Instance with 3% Malfunction

As Figure 3 illustrates, the first reruns have a growing time consumption (in the example up to the 20th rerun), but after some trains have reached their goals, the rerun time decreases as does the window, due to fewer malfunctions. It is still not the most efficient solution, as the incremental approach illustrates, but the scaling is not as bad as expected.

The hypothesis, that the malfunction handling of the Graph approach works worse for malfunction dense environments, did not hold. The hypothesis was, that the extension of the horizon for the Graph, performs worse with more malfunctions. Taking incremental as a comparison, one can see that the graph approach performs better in the dense setting, thus refuting our assumption. We

did not take into account, that more malfunctions also tend to approach the worst case, and thus improve the performance of the approach relatively.

Our assumption that the naive approach would perform worse than the graph approach did not hold. It seems, that computing the graph is similarly as efficient as reading the graph from input, with most instances performing slightly better, if the graph is recomputed (naive). There was no significant difference even on sparse instances. As we can see in the long tail the grounding of the graph is under a second.

But the assumption, of the incremental outperforming the other approaches did hold. It shows shorter runtimes over all instances, also requiring less reruns, as its solutions always use the optimal horizon. The window is shrinking on every rerun and only extended if required. One can spot the reruns, where more timesteps were necessary, by looking at the peaks in Figure 3.

We can see a nice logarithmic curve, due to the nature of NP problems, as each choice prunes the search space.

7 Conclusions

When testing the environment, the incremental approach of using the estimate and incrementing the timesteps when required, outperformed the others on all instances. It also brought more predictable computation times (except the peaks), after the first iteration, which can be useful for real world scenarios. This illustrates the advantages and the ability of an incremental approach to build on and adapt previous knowledge. And it shows such an approach works well, for this kind of problem. A possible extension would be, to pass the grounding information from the previous runs, instead of just passing the representation. In concept, it should be possible as the map does not change, and any malfunction just adds new assumptions. That would be a good topic for more research.

As a side benefit, incremental computes shorter paths (thus needing fewer reruns), as it finds timestep optimal solutions. Optimality, would be a good topic for further study, as enforcing optimality (e.g. trains arrive with as much time left as possible) might further reduce, the necessary window and quicken reruns.

Also, we found that the difference between reading and recalculating the graph is negligible.

Another idea, not explored by this paper, is to use more resilient paths, which ensure a minimum distance between trains, enabling more reusing of previous paths, when reruns happen. It could be interesting to compare whether optimize statements perform significantly worse compared to an incremental solution.

Other than that, all ideas which limit grounding and solving are worth pursuing and applicable to Flatland as long as they are applicable to graph structures.

References

- Gómez et al., 2020. Gómez, R. N., Hernández, C., and Baier, J. A. (2020). Solving sum-of-costs multi-agent pathfinding with answer-set programming. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(06):9867–9874.

- Kaminski et al., 2017. Kaminski, R., Schaub, T., and Wanko, P. (2017). A tutorial on hybrid answer set solving with clingo. *Reasoning Web. Semantic Interoperability on the Web: 13th International Summer School 2017, London, UK, July 7-11, 2017, Tutorial Lectures 13*, pages 167–203.
- Murphy, 2025a. Murphy, R. K. (2025a). Flatland. <https://github.com/krr-up/flatland>. Accessed: 01.03.2025.
- Murphy, 2025b. Murphy, R. K. (2025b). krr-up issue #30. <https://github.com/krr-up/flatland/commits/malfunctions>. Accessed: 04.03.2025.
- Murphy, 2025c. Murphy, R. K. (2025c). Malfunctions commit branch. <https://github.com/krr-up/flatland/commits/malfunctions>. Accessed: 01.03.2025.

A Appendix

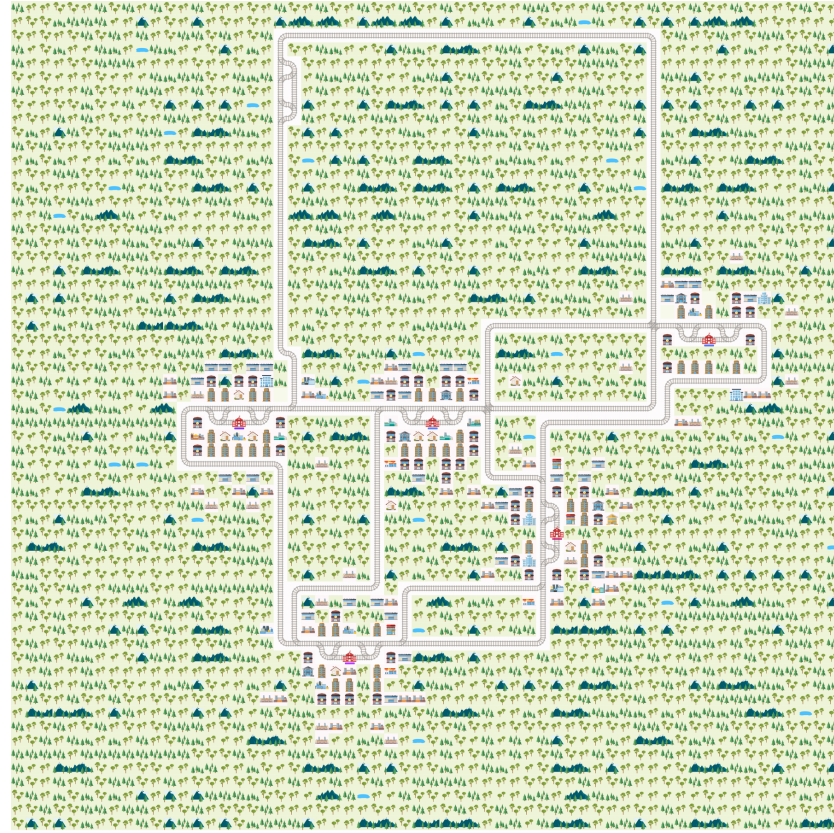


Fig. 4: Sparse instance, using a 60x60 grid, where 6 trains must travel across 6 cities.

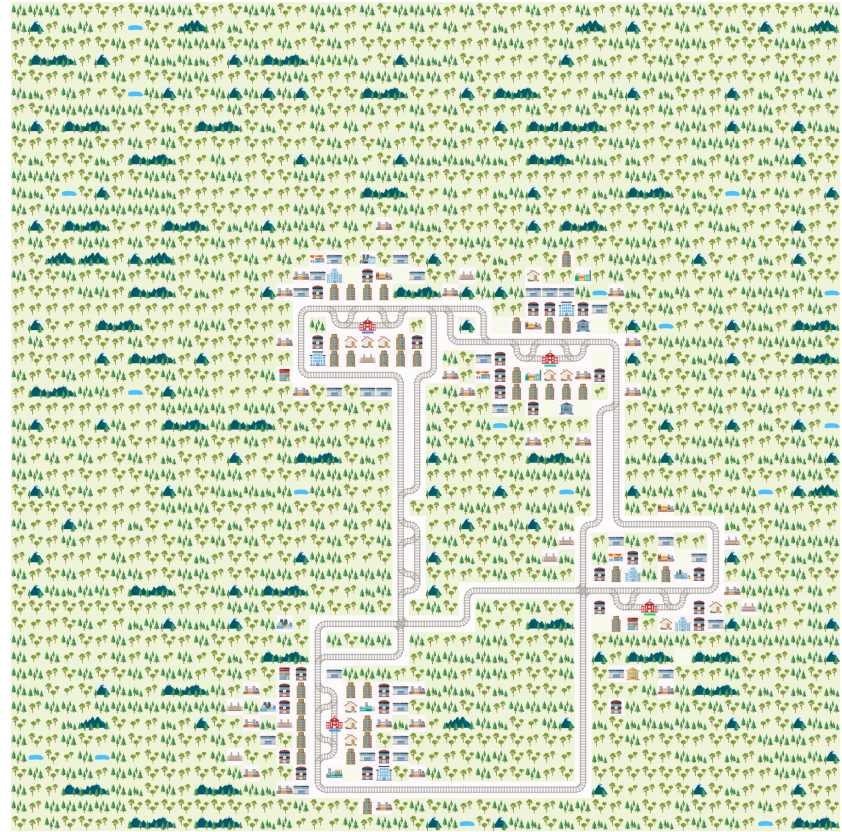


Fig. 5: Medium instance, using a 50x50 grid, where 10 trains must travel across 5 cities.



Fig. 6: Dense instance, using a 24x24 grid, where 20 trains must travel across 3 cities.

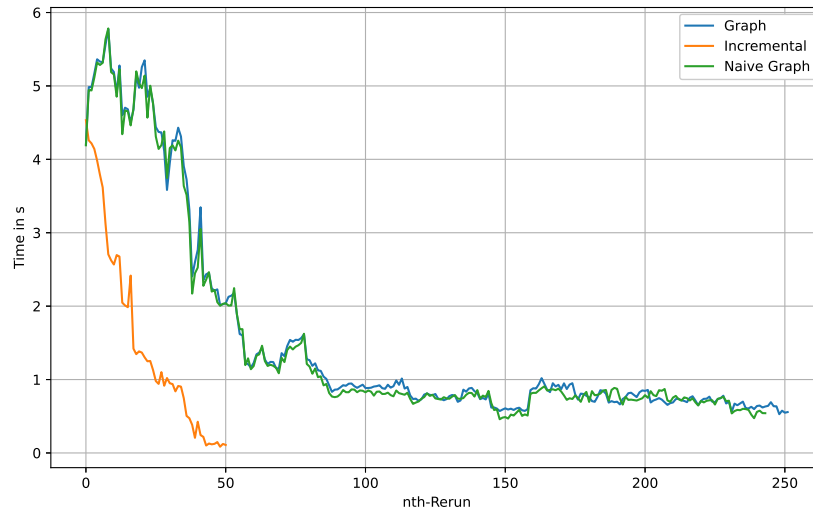


Fig. 7: Full-sized Sparse Benchmark

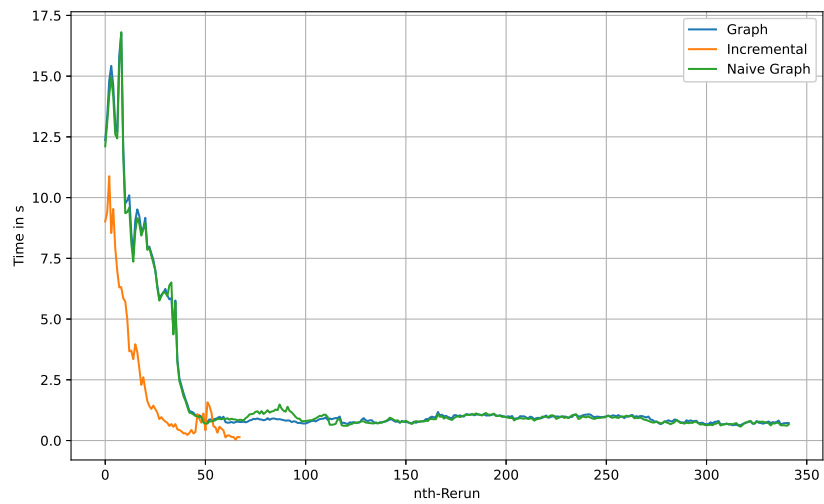


Fig. 8: Full-sized Medium Benchmark

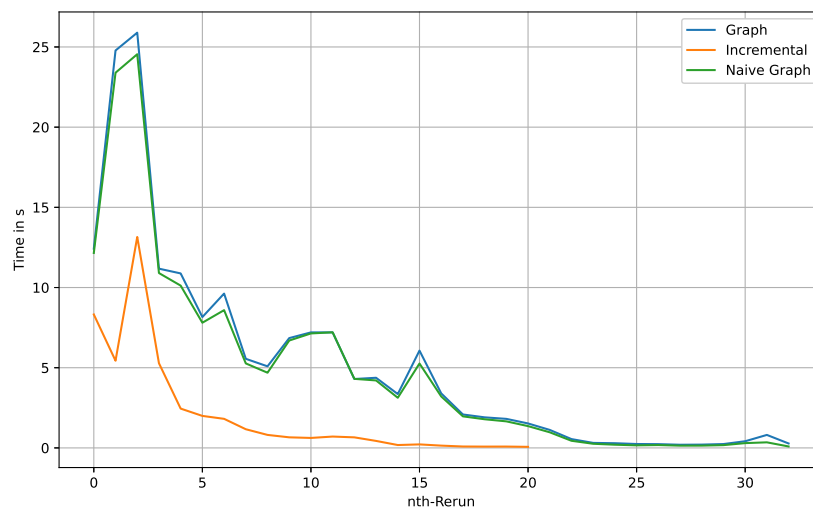


Fig. 9: Full-sized Dense Benchmark